

# Assignment 1

## 1. I2C

I2C는 "Inter-Integrated Circuit"의 약자로, 필립스에서 개발하고 마이크로 컨트롤러와 센서, 메모리, 기타 장치들 간에 단거리 통신을 할 수 있게 해주는 동기식 반이중 직렬 통신 프로토콜이다. I2C 통신은 2개의 선을 이용하는 통신 방식이고 하나의 Master와 다른 하나의 Slave 기기가 물려 통신이 가능하다. 그리고 I2C의 표준 주소 체계는 7비트 주소 방식을 기본으로 하기에 Slave는 127개까지 구성이 가능하다. 하나의 마스터가 여러 Slave 장치를 제어할 수 있으며, 하나의 버스에 여러 마스터가 존재하는 멀티마스터 환경도 가능하다. Clock 신호를 사용하는 동기식 통신 방식이라 시간에 자유롭다. 또한 Slave 선택을 위해 항상 주소 데이터가 붙기에 긴 데이터를 전송하기엔 부적합하다. 통신할 때 사용하는 2개의 선은 SDA(Serial Data Line) 선과 SCL(Serial Clock Line) 선이 있는데, SDA는 데이터를 주고받기 위한 선이고, SCL은 타이밍을 동기화 하기 위한 clock 선이다.

I2C 사용 시에 주의해야 할 점은 "풀업 저항"이다. 이 저항을 다는 이유는 I2C 통신을 위해 SDA 선과 SCL 선이 모두 기본으로 High 상태가 되어야 한다. "풀업 저항"은 이를 High 상태로 만들어 준다. 또한, 하나의 SDA 선을 통해 Tx(송신), Rx(수신)를 처리하기 때문에, 아무도 통신하지 않을 때 Floating 현상이 발생함을 방지하기 위해 "풀업 저항"을 달기도 한다.

데이터를 전송 또는 수신할 건지는 Master에서 주도한다. 그리고 데이터를 송/수신을 할 때 반드시 Slave 주소를 마스터에 명시해 두어야 통신이 된다. I2C는 동기식 반이중 통신 방식으로 반이중이란, 매번 송신, 수신을 따로 전송한다. 즉, 한번에 한 방향씩 만(데이터는 양방향이지만, 동시에는 안됨) 가능하다. 동기식 통신 방식이란, clock 신호를 이용해서 데이터를 얼마나 보내는지 타이밍을 맞춘다. 이는 통신속도를 따로 정해주지 않아도 된다는 점이 있다.

동작 방식은 다음과 같다. I2C 통신은 Master가 클럭을 제어하고, Slave는 이에 따라 데이터를 송/수신한다. 통신 과정은 다음과 같다.

- i) 시작 조건: SDA가 High에서 Low로 떨어질 때, SCL이 High이면 시작 조건으로 인식
- ii) 주소 전송: Master는 7비트(또는 10비트)의 Slave 주소와 Read/Write 비트를 전송
- iii) ACK/NACK 응답: Slave는 자신이 선택된 경우(수신 쪽) ACK(확인응답 신호)를 보냄
- vi) 데이터 전송: 이후 Master와 Slave 간에 8비트 단위의 데이터가 교환되며, 각 바이트마다 ACK 신호가 따라옴
- v) Stop 조건: SDA가 Low에서 High로 상승할 때, SCL이 High이면 통신 종료

## 2. SPI

SPI는 Serial Peripheral Interface의 약자로, 하드웨어(주변 장치) 간의 데이터 교환을 위한 동기식 직렬 통신 프로토콜이다. 주로 Micro controller, Shift register, SD 카드 등의 소형 주변 장치와 데

이터를 송/수신하고, 쉽게 말해서 IC(칩)과 IC 간의 데이터를 주고받는 방식이다.

SPI는 1:N(1 대 다수)의 통신을 지원한다. 한 번에 여러 장치와 데이터를 주고받을 수는 있지만, 그만큼 통신선이 많이 필요하다는 단점이 있다. 동시에 여러 장치와 송/수신이 가능하고 I2C에 비해 속도가 빠르다. 또한, 간단하게 배선이 가능하고 RS-232, CAN 버스보다 비교적 짧은 거리에서 동작한다. 그리고 하나의 Master 장치와 하나 이상의 Slave 기기를 연결해 사용한다.

SPI 통신 시그널에는 송신과 수신을 위한 MOSI(Master Out Slave In)선과 MISO(Master In Slave Out)선, 클럭(SCLK)선, SS(=CS, Slave Select)선 이렇게 4개가 필요하다. 먼저 **MOSI**는 Master Out Slave In 이름 그대로 Master에서 출력하고 Slave에서 입력 받는 선이다(Master → Slave 데이터 전송). **MISO**는 Master In Slave out 이름 그대로 Slave에서 출력하고 Master에서 입력 받는 선이다(Slave → Master 데이터 전송). 보통 MOSI를 통해 Slave에 명령 데이터가 들어오면, MISO를 통해 Slave에서 Master로 응답 데이터가 출력된다. **SS(=CS)**는 Slave Select(=Chip Select)로, 보통 SS 핀이라고도 하며, Master 장치가 여러 개의 Slave 장치와 통신할 때, Slave 하나를 선택하여 통신을 하는데 그때 선택을 하는 역할을 하는 신호선이다. 한마디로 어디로 보낼 지 결정하는 신호선이다. **SCLK(=SCK, CLK)**는 동기화 신호이며, Clock 전송을 위한 단자로, Master에서 Slave로 Clock을 전송한다.

SPI 통신 프로토콜은 먼저 Master에서 어떤 값을 Slave로 보낸다고 가정했을 때, 보내는 1 Clock의 신호마다 1bit의 데이터가 이동된다. Master는 SS(=CS)의 신호에 따라 데이터를 전송하고자 하는 Slave를 선택할 수 있다. 원하는 Slave의 SS에 Low(0)을 주고 다른 Slave들의 SS에는 High(1)을 준다. SPI 장치들은 Shift Register를 가지고 있어, 이를 활용해 데이터를 전송한다. Master에 보내려는 데이터가 있고, Slave에는 같은 크기의 '쓰레기 값'이 저장되어 있다(만약 Slave에서도 데이터를 보내고 싶으면, 같은 크기의 데이터를 저장해주면 된다). 위에서 설명한 것처럼 1 Clock 당 1bit의 데이터가 하나씩 전송되고 shift되고 이렇게 최종 데이터까지 다 보내지면 통신이 종료된다.

## SPI 통신의 장점

1. 완전한 전이중(Full Duplex) 통신 → 양방향 통신
2. 전송되는 비트에 대한 완전한 프로토콜 유연성을 갖춰 최대 16bit까지 마음대로 길이를 조절할 수 있다.
3. 전송기가 필요하지 않다. 흔히 말하는 트랜시버를 사용할 필요가 없다.
4. 매우 단순한 하드웨어 인터페이스 처리로 아주 단순한 센서나 메모리에서 많이 사용한다.
5. IC 패키지에 4개의 핀만 사용한다.
6. 최대 Clock이 제한되지 않아 속도 제한이 없다.
7. Push-Pull 출력(Open Drain이 아닌)을 사용하여 상호 간에 같은 전압을 사용하여 시그널 정합성과 고속을 지원한다.
8. I2C 보다 낮은 소비 전력

9. Slave는 Master가 보내주는 Clock 만을 사용하고 정확성이 떨어져도 문제없다.

### **SPI 통신의 단점**

1. 하드웨어 Slave 인식이 없다.
2. Slave에 의한 하드웨어 흐름제어가 없다.
3. 오류 검사 프로토콜이 정의되어 있지 않다. (Error checking 지원)
4. 노이즈 스파이크에 영향을 받는 경향이 있다.
5. RS-232, CAN 버스보다 비교적 더 짧은 거리에서 동작한다. (칩 간 통신에서만 주로 사용)
6. 하나의 마스터 장치만 지원한다.
7. 인밴드(Default SPI wire)를 통해 주소가 지원되지 않아 다수의 Slave를 사용 시 별도의 아웃밴드(칩 선택 라인)를 통해 Slave를 선택해야 한다.
8. Hot 플러그를 지원하지 않는다.

### **3. SDIO**

SDIO는 Secure Digital Input Output의 약자로, 기존 SD 메모리 카드 슬롯을 통해 저장 기능뿐만 아니라 다양한 입출력 기능을 수행할 수 있도록 확장된 인터페이스이다. 이를 통해 SD 카드 슬롯을 활용하여 무선 LAN, 블루투스, GPS, 카메라 등 다양한 주변 장치를 연결할 수 있다.

SDIO의 특징으로는 최대 7개의 I/O 기능과 1개의 메모리 기능을 제공하여 복합적인 장치 구현이 가능하고 이벤트 기반의 인터럽트를 제공하여 효율적인 데이터 처리가 가능하다. 장치 정보는 튜플(Tuples) 형태로 제공되어 시스템이 장치 특성을 쉽게 파악할 수 있다. 그리고 SDIO는 SD 카드와 동일한 전기적/물리적 인터페이스를 사용하며, SPI 모드와 SD 모드 두 가지 동작 방식을 지원한다.

SDIO 카드의 특징으로는 전압의 범위가 2.7V에서 3.6V 사이에서 동작하며, 다양한 전압 환경에서 사용 가능하다. 또한, 탈착 가능 및 Plug and Play 지원하여 사용자가 필요에 따라 장치를 쉽게 연결하거나 제거할 수 있다. 그리고 다양한 form factor를 가지고 있는데, 표준크기 SDIO, mini SDIO, micro SDIO 등 다양한 형태로 제공되어 다양한 기기에 적용할 수 있다.

임베디드 SDIO 장치의 특징으로는 2.7V - 3.6V 또는 1.7V - 1.96V에서 동작하여 저전력 설계에 유리하며, 비탈착형 장치로 시스템 내부에 고정되어 사용되며, 외부에서 제거할 수 없다. 또한, 유연한 form factor와 핀 레이아웃으로 설계에 따라 다양한 형태와 핀 구성을 가질 수 있다. 그리고 공유 버스 연결이 가능하여 여러 장치가 동일한 버스를 공유하여 시스템 확장성을 높일 수 있다.

SDIO의 동작 방식에는 최대 8개의 버스를 사용하며, 주요 신호 핀은 다음과 같다. CMD 라인, DATA 라인, CLK 라인이 있다. CMD 라인은 SDIO 장치에 명령어 전송을 하고 장치는 이에 대한 응답을 반환하는데 사용되고, DATA 라인은 데이터 전송에 사용되며, 1bit 또는 4bit 모드로 동작

할 수 있고, CLK 라인은 Clock 신호를 제공하여 동기화를 담당한다. 이러한 구조는 I2C나 SPI 유사하게 Clock 신호를 기준으로 명령어를 보내고 데이터를 읽거나 쓰는 방식으로 동작한다.

SDIO의 장단점으로는

장점은 SD 슬롯을 활용하여 기존 SD 카드 슬롯을 그대로 이용 가능하고 최대 수십 Mbps까지 가능하며 고속으로 데이터가 전송 가능하다. 또한, 다양한 주변장치를 지원하여 모듈 확장성이 높고 전용 포트 없이 기능 확장이 가능하여 하드웨어를 간소화할 수 있다.

단점은 장치 드라이버 및 명령어 체계 구현이 필요하여 소프트웨어가 복잡하고 일부 SD 슬롯은 SDIO 장치를 지원하지 않아 호환성 문제가 있을 수 있다. 또한, 한 슬롯에 하나의 장치만 연결 가능하며 병렬 연결이 어렵고 고속 통신 및 고성능 모듈일수록 소비 전력이 커서 전력 소비가 커진다.

#### **4. PCI**

PCI는 Peripheral Component Interconnect의 약자로, 컴퓨터 내부 장치들이 서로 데이터를 빠르게 주고받을 수 있도록 설계된 하드웨어 버스 아키텍처이다. PC의 CPU는 32bit로 넘어갔지만 메인보드의 확장 슬롯은 16bit를 사용하는 ISA에 한동안 머물러 있었는데, 486 시대에 이르러서 확장카드의 스펙이 올라가면서 ISA의 속도로는 한계가 오기 시작했다. 따라서 PCI는 ISA 버스를 대체하는 목적으로 나왔다. PCI는 인텔이 개발했고, 주로 그래픽 카드, 사운드 카드, 네트워크 카드, 저장장치 컨트롤러 등 다양한 하드웨어 장치를 마더보드(motherboard)에 연결할 때 사용되었다.

구조와 특징으로는 병렬 인터페이스, PnP(Plug and Play), 버스 마스터링, 공유 버스 구조 등이 있다. 병렬 인터페이스에서 PCI는 병렬 방식의 데이터 전송을 사용하여, 한번에 여러 bit를 전송할 수 있다. 일반적으로 32bit 또는 64bit 버스 폭을 가지며, Clock 속도는 33MHz or 66MHz이다. PnP는 운영체제가 PCI 장치를 자동으로 인식하고 설정할 수 있도록 해주어, 사용자가 복잡한 설정을 직접 하지 않아도 된다. 버스 마스터링은 장치가 CPU의 개입 없이도 메모리에 직접 접근할 수 있도록 지원하여, 성능을 향상시킨다. 공유 버스 구조는 여러 장치가 하나의 버스를 공유한다. 이를 통해 확장성이 있지만, 동시에 여러 장치가 사용할 경우 병목 현상이 발생할 수 있다.

발전 과정으로는 PCI는 1990년대부터 2000년대 초반까지 표준 인터페이스로 사용되었다. 하지만 데이터 전송 속도가 향상되고 고속 통신이 요구됨에 따라 PCI-X, AGP, 그리고 PCI Express(PCIe)로 진화됐다. PCI-X(PCI Extended)는 서버용으로 개발되었고 더 높은 대역폭을 제공한다.

AGP(Accelerated Graphics Port)는 그래픽 카드 전용 슬롯으로, PCI보다 빠른 속도를 제공한다. PCI Express(PCIe)는 직렬 통신을 기반으로 하며, 매우 높은 속도와 유연한 구조를 제공하여 현재는 가장 널리 사용되는 확장 버스이다.

PCI 슬롯은 데스크톱 컴퓨터의 마더보드에 위치하며, 다양한 주변 장치를 꽂아 사용할 수 있다. 예를 들어, 구형 컴퓨터의 무선 랜카드, 사운드 카드를 추가하여 고급 오디오 지원, RAID 컨트롤러 카드로 저장 장치 관리 등이 있다.

현재 대부분의 최신 컴퓨터 시스템에서는 PCI Express가 PCI를 대체하고 있다. 그러나 일부 산업용 시스템이나 구형 장비에서는 여전히 PCI 슬롯과 장치가 사용되고 있다.

## **5. USB**

USB는 Universal Serial Bus의 약어로, 컴퓨터와 주변 기기 간의 데이터 전송 및 전원 공급을 위한 표준 인터페이스이다. 이는 아키텍처, 특히 개인용 컴퓨터와 같은 호스트와 주변 장치(ex: 컴퓨터 등)와의 데이터 전송 및 전력 공급을 위한 물리적 인터페이스 및 통신 프로토콜을 규정한다. 1996년 처음 등장한 이후, PnP, 핫 플러깅, 범용성 등의 장점으로 인해 컴퓨터, 스마트폰, 가전제품 등 다양한 기기에서 가장 널리 사용되는 인터페이스가 되었다.

USB는 기존의 다양한 포트(시리얼, 패러렐, PS/2 등)를 하나로 통합하려는 목적에서 개발되었다. 초기에는 키보드, 마우스, 프린터 등을 연결하기 위한 용도였지만, 지금은 저장 장치, 카메라, 스마트폰, VR 장비 등 다양한 분야로 확장되었다.

주요 특징으로는 PnP, 핫 플러깅(Hot Plugging), 데이터 전송과 전원 공급 동시 지원, 범용성 등이 있다. PnP는 장치를 연결하면 자동으로 인식되어 설치가 간편하다. 핫 플러깅은 컴퓨터가 켜진 상태에서 장치를 안전하게 연결하거나 제거할 수 있다. 데이터 전송과 전원 공급 동시 지원은 하나의 케이블로 데이터 전송과 전원 공급이 동시에 이루어진다. 범용성은 다양한 제조사, 플랫폼에서 호환되어 폭넓은 사용이 가능하다.

USB의 종류에는 USB-A, USB-B, USB-C, Micro-USB/Mini-USB 등이 있다. USB-A는 가장 보편적인 형태로, 일반 컴퓨터에 흔히 사용된다. USB-B는 주로 프린터 등 주변 기기에 사용된다. USB-C는 양면으로 사용이 가능하고 고속 데이터 및 전력 전달 지원을 한다(차세대 표준). Micro-USB/Mini-USB는 모바일 기기 및 구형 장치에서 사용된다.

USB 버전은 크게 USB 1.1, USB 2.0, USB 3.0, USB 3.1, USB 3.2/4.0이 있다. USB 1.1은 1998년에 발표하고 최대속도는 12 Mbps이다. 이는 초기 버전으로 기본적인 주변기기 연결을 한다. USB 2.0은 2000년에 발표하고 최대속도는 480Mbps이다. 이는 대중화의 시작이 됐고, 현재도 널리 사용된다. USB 3.0은 2008년에 발표하고 최대속도는 5Gbps이다. 이는 파란색 포트이고, 고속 데이터 전송 지원이 된다. USB 3.1은 2013년에 발표하고 최대속도는 10Gbps이다. 이는 향상된 속도를 지니고, USB-C와 함께 사용된다. USB 3.2/4.0은 2017/2019년에 발표하고 최대속도는 20/40Gbps이다. 이는 썬더볼트3과 호환 가능하고 고성능 장비 지원을 한다.

## **6. Bluetooth**

Bluetooth는 근거리 무선 통신 기술 중 하나로, 짧은 거리(보통 10m이내)에서 전자기기 간의 데이터 교환을 가능하게 한다. 1994년 에릭슨이 처음 개발했고, 이후 다양한 기업들이 참여하여 Bluetooth Special Interest Group(SIG)이 설립되며 표준화가 이루어졌다. 오늘날 스마트폰, 무선 이어폰, 키보드, 웨어러블 기기, 차량 등 수많은 디지털 장치에서 사용된다.

블루투스의 무선 시스템은 ISM(Industrial Scientific and Medical) 주파수 대역인 2400~2483.5MHz를 사용하고 있다. 이중에서 위 아래 주파수를 쓰는 다른 시스템의 간섭을 막기 위해서 2400MHz 이후 2MHz, 2483.5MHz 이전 3.5MHz까지의 범위를 제외한 2400MHz 이후 2483.5MHz 이전 3.5MHz까지의 범위를 제외한 2402~2480MHz, 총 79개 채널을 사용하고 있다. 여기서 ISM이란 산업, 과학, 의료용으로 할당된 주파수 대역으로서 전파 사용에 대한 허가가 필요없는 개인 무전기에 많이 사용하고 있고 아마추어 무선, 무선랜, 블루투스가 이 ISM에 해당한다. 블루투스는 같은 주파수 대역을 사용하기에 서로 간의 전파 간섭이 생길 수 있는데 이를 예방하기 위해서 “주파수 호핑(Frequency Hopping)”을 사용한다. “주파수 호핑”이란 많은 수의 채널을 특정 패턴에 따라서 빠르게 이동하여 간섭을 최소화하여 데이터를 조금씩 전송하는 방식이다. 블루투스는 기기 간에 동기화가 되어야만 작동하게 되어 있는데 Master와 Slave 구성으로 연결된다. Master와 Slave와의 동기화 이외에 Slave-Slave간의 통신은 불가능하다.

Bluetooth의 버전에는 1.0~2.0, 2.1+EDR, 3.0, 4.0, 5.0, 5.1~5.3이 있다. 1.0~2.0 버전은 1999~2004년에 발표됐고, 기본 무선 통신과 최대 1 Mbps의 속도를 가진다. 2.1+EDR 버전은 2007년에 발표됐고, 페어링 간소화와 전송속도 향상이 되었다. 3.0 버전은 2009년에 발표됐고, 고속 전송(최대 24Mbps, Wi-Fi 병용)이 가능해졌다. 4.0 버전은 2010년에 발표됐고, BLE(Bluetooth Low Energy)가 도입되었다. 5.0 버전은 2016년에 발표됐고, 범위확대(최대 240m)와 속도 향상(2Mbps)이 되었다. 5.1~5.3 버전은 2019년~에 발표됐고, 방향 탐지와 위치기반 서비스 강화, 그리고 더 낮은 전력 소모가 된다.

Bluetooth의 장점은 설치가 간편하고 별도의 네트워크가 필요 없다. 또한 다양한 기기와의 호환성과 저렴한 비용으로 낮은 전력이 소모된다. 단점으로는 상대적으로 낮은 전송 속도(Wi-Fi 대비)와 거리 제한 및 장애물의 영향을 받는다. 또한 일부 환경에서는 간섭 문제가 발생할 가능성이 있다.

## **7. CAN**

CAN(Controller Area Network)이란, 차량 내에서 호스트 컴퓨터 없이 Micro Controller나 장치들이 서로 통신하기 위해 설계된 표준 통신 규격이다. 차량 내 ECU(Electronic Control Unit)들은 CAN 프로토콜을 사용하여 통신한다. 초기에는 차량 네트워크용으로 개발되었으나 최근에는 차량뿐만 아니라 산업 전 분야에 폭넓게 적용되고 있다. 현대의 자동차에는 수많은 ECU(전자제어 장치)가 존재한다. 예를 들어, 엔진, 브레이크, 조향, 에어백 센서, 인포테인먼트 시스템 등이 각각의 ECU를 통해 작동한다. 과거에는 UART, 즉 이들을 하나씩 연결하는 방식(point-to-point)으로 제어했지만, 배선이 복잡하고 효율성이 떨어졌다. 이에 따라 하나의 버스를 공유하면서 통신하는 방식인 CAN이 등장하여 시스템을 간결하고 효율적으로 만들었다.

CAN 통신은 여러 개의 ECU를 병렬로 연결하여 데이터를 주고받는 통신방법이다. CAN 버스를 통해서 통신선 상에 데이터를 띄워놓고 필요한 데이터에 접근한다. CAN 통신의 특징은 다음과 같다. 메시지 지향성 프로토콜, 보완적인 에러 감지 메커니즘, 멀티마스터 능력, 결점 있는 노드의 감지와 비활성화, 전기적 노이즈에 강함, 저렴한 가격 및 구성의 용이성, 고속 및 원거리 통신, PnP 기능 등이 있다. 메시지 지향성 프로토콜의 의미는 CAN에서 노드의 주소에 의해 데이터가

교환되는 것이 아니라 메시지의 우선순위에 따라 ID를 할당하고, 이 ID를 이용해 메시지를 구별하는 방식을 사용한다. 보완적인 에러 감지 메커니즘의 의미는 CAN에서는 다양한 에러 감지 메커니즘이 상호 보완적으로 에러를 감지하기 때문에 높은 안정성을 보장한다. 또한, 메시지 전송 시 에러가 감지되면 자동적으로 해당 메시지를 즉시 재전송하므로 에러 회복 시간이 짧다. 멀티마스터 통신 방식은 CAN을 기반으로 한 네트워크에는 버스를 점유하기 위한 감독자 노드(Bus Master)의 필요가 없다. 즉, 모든 노드가 버스 마스터가 되어 버스가 비어 있을 때(idle)라면 언제든지 메시지 전송이 가능하다. 모든 노드는 버스가 비워지는 즉시 메시지 전송을 시작한다. 전기적 노이즈에 강하다는 것은 꼬인 2선(Twist Pair Wire, CAN\_High, CAN\_Low)을 이용하여 전기적으로 차별되는 통신을 하여 전기적 노이즈에 매우 강하다.

CAN의 통신방식은 비트 단위로 충돌을 감지하고 회피하는 CSMA/CR(Carrier Sense Multiple Access/Collision Resolution) 방식을 사용한다. 각 메시지에는 고유의 ID가 있으며, 이 ID는 우선순위 역할도 한다. 전송속도는 표준 CAN 기준으로 최대 1Mbps의 속도를 가진다. 프레임 구조는 4가지로, 데이터 프레임, 원격 프레임, 오류 프레임, 오버로드 프레임이 존재한다.

CAN의 종류에는 11비트 식별자를 사용하는 Standard CAN(2.0A), 29비트 식별자를 사용하는 Extended CAN(2.0B), 그리고 데이터 길이 및 속도 향상(최대 64byte, 더 빠른 속도)이 된 CAN FD(Flexible Data-rate)이 존재한다.

## **8. UART**

UART(Universal Asynchronous Receiver/Transmitter)는 직렬 통신 방식 중 하나로, 두 장치 간 데이터를 비동기식으로 송/수신하기 위해 사용되는 하드웨어 모듈 또는 통신 프로토콜이다. 컴퓨터, 마이크로 컨트롤러, 센서, GPS 모듈 등에서 사용되며, 임베디드 시스템에서 매우 중요한 역할이다. UART는 데이터 송신 측에서는 데이터를 비트 단위로 직렬화(Serializing)하여 전송하고, 수신 측에서는 이를 병렬화(Deserializing)하여 원래의 데이터로 복원하는 역할을 한다. 비동기식 통신이므로, 수신 쪽에서 동기신호를 찾아내어 데이터의 시작과 끝을 시간적으로 알아 처리할 수 있도록 보내는 쪽(Tx)과 받는 쪽(Rx)에서 약속을 정해야 하는데, 이를 프로토콜이라 한다. 또한, 송신기와 수신기가 동일한 통신 속도(baud rate)로 작동해야 한다. 예를 들어 마이크로컨트롤러↔컴퓨터, 센서↔MCU가 있다. UART에서는 보내는 쪽(Tx)과 받는 쪽(Rx)에서 데이터를 보내는 속도를 보율(baud rate)로 정하고 있다. 1초 당 얼마나 많은 데이터를 보내냐는 것을 의미한다. 예를 들어, 신호가 1bit인데 “9600 Baud rate”라고 하면 1초에 9600개의 데이터를 송신 또는 수신할 수 있는 것이다. 즉, 서로의 통신 속도를 일정하게 해서 통신을 하게 된다. 같은 데이터를 전송할 지라도 전송 속도에 따라 받는 데이터가 달라질 수 있다.

비동기 통신은 별도의 Clock 신호가 전송되지 않는다. 따라서 데이터의 시작과 끝을 알기 위해, UART에서는 '0'의 시작 비트(start bit)와 '1'의 정지 비트(stop bit)를 사용한다. 시작 비트(Start bit, Low로 전환)는 통신의 시작을 의미하며 한 비트 시간 길이만큼 유지한다. 지금부터 정해진 약속에 따라 통신을 시작한다. 데이터 비트(Data bit)는 5~8비트의 데이터 전송을 하고, 몇 비트를 사용할 것인지는 해당 레지스터 설정에 따라 결정된다. 패리티 비트(Parity bit)는 오류 검증을 하기

위한 패리티 값을 생성하여 송신하고 수신 쪽에 오류 판단한다. None, Even, Odd 패리티 등의 3가지 옵션으로 해당 레지스터 설정에 따라 선택할 수 있다. 'None'을 선택하면 이 비트가 제거된다. 정지 비트(Stop bit, High 상태)는 통신 종료를 알린다. 3가지의 정해진 비트만큼 유지해야 한다. 1, 1.5, 2비트로 해당 레지스터 설정에 따라 결정된다. 송신기와 수신기는 Baud rate(초당 전송 속도), Data bit, Parity bit, Stop bit 설정을 동일하게 맞춰야 통신이 정상적으로 이뤄진다.

UART의 주요 특징으로는 두 장치 간 직접 연결하는 1:1 통신 방식, 별도의 Clock 신호 없이 동작하는 비동기식 전송(Start/Stop bit로 타이밍 동기화), 전송 선(Tx)과 수신 선(Rx) 2개의 선으로 구성된 간단한 구조 등이 있다.

## **9. GPIO**

GPIO는 General Purpose Input/Output의 약자로, 마이크로컨트롤러(MCU), 마이크로프로세서(MPU), 라즈베리파이(Raspberry Pi) 등 임베디드 시스템에서 사용되는 범용 입출력 핀이다. 이름 그대로, 특정한 용도에 제한되지 않고 사용자가 원하는 용도로 설정할 수 있는 핀을 의미한다. GPIO 핀은 2가지 방식으로 사용할 수 있다. 입력(input)은 버튼, 센서 등의 외부 신호를 읽기 위한 용도이고 출력(output)은 LED, 모터, 릴레이 등의 외부 장치를 제어하기 위한 용도이다. 사용자는 소프트웨어를 통해 각 핀의 모드를 설정하고 동작을 제어할 수 있다.

GPIO의 주요 기능으로는 High(1) 또는 Low(0) 상태를 주고받는 디지털 입출력, 입력 혹은 출력으로 자유롭게 설정 가능한 핀 방향 설정, 입력 상태에서 핀의 부동 상태를 방지하는 풀업(Pull-up)/풀다운(Pull-down) 저항 설정, 입력 변화(상승/하강 엣지)를 감지해 특정 동작 수행을 가능하게 하는 인터럽트 설정, PWM(Pulse Width Modulation) 기능 지원(일부 핀만 해당) 등이 있다. 주의사항으로는 전압 제한, 입력 핀 부동(Floating) 방지, 전류 제한, 쇼트 방지 등이 있다. 전압 제한은 대부분의 GPIO 핀은 3.3V 또는 5V만 허용한다. 그래서 초과 시 핀 손상 위험이 있다. 입력 핀 부동 방지는 풀업/풀다운 저항 설정이 필요하다는 것이다. 전류 제한은 출력 전류가 작기 때문에, 큰 부하(모터 등)는 트랜지스터나 릴레이로 중계한다. 쇼트 방지는 출력 핀끼리 직접 연결하면 단락 위험이 있다는 것이다.

GPIO 설정 방법(일반적인 MCU 기준)은 다음과 같다. 해당 핀을 GPIO 모드로 설정 → 방향 설정(입력 or 출력) → 초기 상태 설정 → 입/출력 동작 수행 → 필요 시 인터럽트 등록. 대부분의 마이크로컨트롤러나 보드에는 레지스터나 라이브러리(API)를 통해 GPIO를 제어한다.

GPIO의 목적은 단순 입출력 제어이며, 설정은 자유롭게 가능하고, 속도는 빠르고, 연결 수는 핀 수에 따라 제한된다.

## Assignment 2

### 1. ar → create, modify, and extract from archives

정적 라이브러리는 컴파일이 된 오브젝트 파일들이 하나의 아카이브(archive)로 묶여 있는 형태로, 오브젝트 파일들을 묶어주는 명령어가 'ar' 명령어이다. 사실 ar 명령어는 정적 라이브러리에 국한되지 않고, 범용적인 비압축 아카이브로 사용할 수 있으나 일반적으로 정적 라이브러리를 다루기 위해 사용된다.

#### 1-1. 아카이브를 생성할 경우

ar rcus [라이브러리 이름] [오브젝트 파일들]

r: 새로운 오브젝트 파일이면 추가, 기존 파일이면 치환

c: 아카이브(라이브러리 파일) 생성, 존재하지 않는 아카이브를 작성(or 갱신)하는 경우에도 경고 메시지를 출력하지 않음

u: 오브젝트 파일의 타임스탬프를 비교해 새로운 파일인 경우에만 치환

s: 아카이브 인덱스 생성. 아카이브 인덱스를 생성하지 않으면 링크 속도가 느려지고, 시스템 환경에 따라 에러가 발생할 수도 있음

```
user@ubuntu:~/test$ vi add.c
user@ubuntu:~/test$ vi add.c
user@ubuntu:~/test$ vi sub.c
user@ubuntu:~/test$ vi main.c
user@ubuntu:~/test$ gcc -c add.c
user@ubuntu:~/test$ gcc -c sub.c
user@ubuntu:~/test$ ls
add.c add.o main.c sub.c sub.o
user@ubuntu:~/test$ ar rcs libmath.a add.o sub.o
user@ubuntu:~/test$ ls
add.c add.o libmath.a main.c sub.c sub.o
user@ubuntu:~/test$ gcc main.c -L. -lmath -o test_program
user@ubuntu:~/test$ ls
add.c add.o libmath.a main.c sub.c sub.o test_program
user@ubuntu:~/test$ ./test_program
add(10, 3) = 13
sub(10, 3) = 7
```

먼저 간단한 C파일(add.c, sub.c) 2개를 만들고 컴파일 해서 오브젝트 파일(.o)을 만든다. 이는 ls 명령어를 통해서 확인할 수 있다. 이후 ar 명령어와 옵션들을 사용해서 libmath 라는 아카이브(라이브러리)를 만든다. 그리고 main.c를 라이브러리와 함께 컴파일을 한다. 이때, -L은 "현재 디렉토리에서 라이브러리를 찾겠다"는 의미이고, -lmath는 "libmath.a를 링크하겠다"는 의미이다. 이후에 실행파일인 test\_program을 실행하면 add와 sub의 출력 결과가 나오는 것을 확인할 수 있다.

## 1-2. 기존 아카이브 파일에서 오브젝트 파일 제거

`ar ds [라이브러리 이름] [오브젝트 파일들]`

d: 아카이브 모듈을 삭제, 삭제할 파일이 없다면 동작하지 않음

```
user@ubuntu:~/test$ ar d libmath.a sub.o
user@ubuntu:~/test$ ls
add.c  add.o  libmath.a  main.c  sub.c  sub.o  test_program
```

ar 명령어에 d 옵션을 붙여서 libmath.a 아카이브에서 sub.o 오브젝트 파일을 삭제한다.

## 1-3. 아카이브의 내용 조회

`ar tv [라이브러리 이름]`

t: 아카이브에 있는 파일 리스트 출력

v: 자세한 내용을 보여주는 verbose 모드로, 이 옵션을 이용하면 파일 크기나 갱신 시각 등의 상세 정보도 출력

```
user@ubuntu:~/test$ ar tv libmath.a
rw-r--r-- 0/0 1232 Dec 31 16:00 1969 add.o
```

위의 ar d 명령어를 통해 아카이브에서 sub.o가 삭제된 것을 확인하려면 ar 명령어에 tv 옵션을 붙여서 확인할 수 있다. 위의 사진을 통해 add.o만 아카이브에 있는 것을 확인할 수 있다.

## 1-4. 아카이브에서 파일 추출

`ar xv [라이브러리 이름]`

x: 아카이브에서 오브젝트 파일 추출

v: 추출한 파일의 파일명 출력

```
user@ubuntu:~/test$ ls
add.c  add.o  libmath.a  main.c  sub.c  sub.o  test_program
user@ubuntu:~/test$ rm *.o
user@ubuntu:~/test$ ls
add.c  libmath.a  main.c  sub.c  test_program
user@ubuntu:~/test$ ar xv libmath.a
x - add.o
x - sub.o
user@ubuntu:~/test$ ls
add.c  add.o  libmath.a  main.c  sub.c  sub.o  test_program
```

위 사진은 현재 디렉토리에 어떤 것이 있는 지 확인한 후에 rm 명령어를 통해서 .o 확장자를 가진 오브젝트 파일들을 모두 삭제한다. 이후 ls 명령어를 통해 삭제된 것을 확인한 후에, libmath.a 아카이브 파일에서 오브젝트 파일을 추출한다. 위 사진에서 보듯이, add.o와 sub.o 두 파일이 아카이브에 있었고 잘 추출된 것을 확인할 수 있다. 이후에 추출된 add.o와 sub.o가 디렉토리에 다시 생긴 것도 확인할 수 있다.

## 2. as → the portable GNU assembler

as는 GNU assembler로 어셈블리 소스파일(.s)을 오브젝트 파일(.o)로 컴파일 해주는 명령어이다.

```
.section .data
msg:
    .asciz "Hello, world!\n"

.section .text
.global _start

_start:
    mov $1, %rax    # write syscall
    mov $1, %rdi    # stdout
    mov $msg, %rsi   # message addr
    mov $14, %rdx   # message length
    syscall

    mov $60, %rax   # exit syscall
    xor %rdi, %rdi  # return cod 0
    syscall
```

.section .data를 통해 데이터 섹션을 정의하고, msg는 문자열이 저장될 레이블(이름)이다. .asciz는 null-terminated 문자열(C 스타일 문자열) → 실제 길이는 14(Wn과 W0 포함)이다. .section .text는 코드(명령어)가 들어갈 영역이고, .global \_start는 Linker가 프로그램 시작점으로 인식하도록 start를 전역으로 선언한다. \_start는 프로그램의 진입점이다.

```
user@ubuntu:~/test2$ vi hello.s
user@ubuntu:~/test2$ as -o hello.o hello.s
user@ubuntu:~/test2$ ld -o hello hello.o
user@ubuntu:~/test2$ ./hello
Hello, world!
```

vi 편집기를 통해 hello.s를 통해 어셈블리 소스파일을 만든다. 그리고 as 명령어를 통해 어셈블리 파일(hello.s)을 오브젝트 파일(hello.o)로 변환한다. 그리고 ld 명령어로 오브젝트 파일을 링크해서 실행 파일(hello)을 생성한다. 마지막으로 실행 파일(hello)을 실행시켜서 결과를 확인하면, 정상적으로 write syscall을 통해 출력을 성공한 것을 확인할 수 있다.

### 3. g++ → GNU C++ compiler

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, world from C++!" << endl;
    return 0;
}
```

```
user@ubuntu:~/test3$ vi hello.cpp
user@ubuntu:~/test3$ g++ hello.cpp -o hello
user@ubuntu:~/test3$ ./hello
Hello, world from C++!
```

g++은 C++ 컴파일러로, .cpp 파일을 컴파일해서 실행 파일을 만들 때 사용한다. g++ hello.cpp -o hello에서 hello.cpp는 입력 소스 파일이고 -o hello는 출력 실행 파일 이름을 hello로 지정해주는 역할이다. 이후에 생성된 실행 파일인 hello를 실행하면 hello.cpp의 내용이 출력되는 것을 확인할 수 있다.

-E: 전처리만 수행 (compile은 안함)

```
user@ubuntu:~/test3$ g++ -E hello.cpp -o hello.i
user@ubuntu:~/test3$ ls
hello.cpp  hello.i
```

위 사진은 C++ 컴파일러인 g++에 -E 옵션을 붙여 .cpp 파일을 .i 파일로 전처리 결과 파일을 생성한 것을 확인할 수 있다.

-S: compile까지만 수행 (어셈블리 코드 생성)

```
user@ubuntu:~/test3$ g++ -S hello.cpp -o hello.s
user@ubuntu:~/test3$ ls
hello.cpp  hello.i  hello.s
```

위 사진은 C++ 컴파일러인 g++에 -S 옵션을 붙여 .cpp 파일을 .s 파일로 어셈블리 코드로 변환한 것을 확인할 수 있다.

-c: 컴파일 + 어셈블 (실행 파일(링크)은 하지 않음)

```
user@ubuntu:~/test3$ g++ -c hello.cpp -o hello.o
user@ubuntu:~/test3$ ls
hello.cpp  hello.i  hello.o  hello.s
```

위 사진은 C++ 컴파일러인 g++에 -c 옵션을 붙여 .cpp 파일을 .o 파일인 오브젝트 파일을 생성한 것을 확인할 수 있다.

-o <file>: <file> 이름의 결과물 만들기

```
user@ubuntu:~/test3$ g++ hello.cpp -o hello
user@ubuntu:~/test3$ ls
hello hello.cpp hello.i hello.o hello.s
```

위 사진은 C++ 컴파일러인 g++을 사용해서 .cpp 파일을 .o 오브젝트 파일로 변환 후 실행 파일을 생성하는 것을 확인할 수 있다.

#### 4. gcc → GNU C compiler

```
#include <stdio.h>

int main() {
    printf("Hello, world form C!\n");
    return 0;
}
```

위 사진과 같은 hello.c 파일이 있고, 아래는 이를 gcc로 컴파일 하는 과정이다.

hello.c → 전처리기 실행 → hello.i(전처리가 완료된 파일) → 컴파일러 실행 → hello.s(기계어로 번역완료(어셈블리어)) → 어셈블러 실행 → hello.o(기계어로 된 오브젝트 파일) → 링커 실행 → hello(실행파일)

```
user@ubuntu:~/test4$ gcc -E hello.c -o hello.i
user@ubuntu:~/test4$ ls
hello.c hello.i
```

-E 옵션이 붙어 전처리기가 실행이 되어 hello.i라는 전처리가 완료된 파일을 생성된 것을 확인할 수 있다.

```
user@ubuntu:~/test4$ gcc -S hello.i -o hello1.s
user@ubuntu:~/test4$ ls
hello1.s hello.c hello.i
```

-S 옵션이 붙어 컴파일러가 실행이 되어 hello1.s라는 기계어로 번역이 완료된 어셈블리어 파일이 생성된 것을 확인할 수 있다.

```
user@ubuntu:~/test4$ gcc -c hello1.s -o hello2.o
user@ubuntu:~/test4$ ls
hello1.s hello2.o hello.c hello.i
```

-c 옵션이 붙어 어셈블러가 실행이 되어 hello2.o라는 기계어로 된 오브젝트 파일이 생성된 것을 확인할 수 있다.

```
user@ubuntu:~/test4$ gcc -g hello2.o -o hello
user@ubuntu:~/test4$ ls
hello hello1.s hello2.o hello.c hello.i
```

-g: 디버깅을 위한 flag를 심어 컴파일

-g 옵션이 붙어 링커가 실행이 되어 오브젝트 파일을 hello라는 실행 파일로 만든 것을 확인할 수 있다.

```
user@ubuntu:~/test4$ ./hello
Hello, world form C!
```

생성된 hello라는 실행 파일을 실행시키면 hello.c 파일에 있는 내용을 출력하는 것을 확인할 수 있다.

## 5. ld → The GNU linker

ld는 링커로, 여러 개의 오브젝트 파일(.o)을 묶어서 하나의 실행 파일로 만든다. C나 C++로 컴파일을 하면, gcc/g++이 내부적으로 ld를 자동 호출하지만, ld를 직접 쓰면 링크 과정을 완전히 수동으로 제어할 수 있다.

```
.section .data
msg:
    .asciz "Hello from ld!\n"

.section .text
.global _start

_start:
    mov $1, %rax    # write syscall
    mov $1, %rdi    # stdout
    mov $msg, %rsi  # message addr
    mov $16, %rdx   # message length
    syscall

    mov $60, %rax   # exit syscall
    xor %rdi, %rdi  # return cod 0
    syscall
```

```
user@ubuntu:~/test5$ vi hello.s
user@ubuntu:~/test5$ as -o hello.o hello.s
user@ubuntu:~/test5$ ld -o hello hello.o
user@ubuntu:~/test5$ ls
hello  hello.o  hello.s
user@ubuntu:~/test5$ ./hello
Hello from ld!
```

왼쪽 사진을 보면 hello.s로 어셈블리 코드를 작성한 것이다. 이제 as 명령어로 hello.s 어셈블리 코드를 hello.o 오브젝트 파일로 만든다. 이제 ld 명령어를 사용해서 hello.o 오브젝트 파일을 hello 실행 파일로 만든다. 생성된 hello 실행 파일을 실행하면 어셈블리 코드에 적힌 메시지 값이 출력되는 것을 확인할 수 있다. 이 모든 내용이 C 코드에서도 물론 가능하다.

## 6. nm → list symbols from object files

nm 명령어는 오브젝트 파일(.o), 실행 파일, 라이브러리(.a), 공유 라이브러리(.so) 등 안에 어떤 심볼이 들어 있는지 확인할 수 있다.

```
#include <stdio.h>

int globalVar = 10;

int add(int a, int b) {
    return a + b;
}

int main() {
    printf("Hello, world!\n");
    return 0;
}
```

```

user@ubuntu:~/test6$ gcc -c main.c -o main.o
user@ubuntu:~/test6$ nm main.o
0000000000000000 T add
                  U _GLOBAL_OFFSET_TABLE_
0000000000000000 D globalVar
0000000000000014 T main
                  U puts

```

위 사진은 작성된 main.c c 파일을 main.o 오브젝트 파일로 변환한 후에 nm 명령어로 어떤 심볼이 있는 지 확인하였다. 사진을 보면 T, U, D가 있다. T는 text section에 있는 심볼(code)로 정의된 함수이다. U는 Undefined로 외부 심볼을 의미한다. D는 data section에 있는 정의된 전역 변수로 심볼이 data section에 위치한다.

```

user@ubuntu:~/test6$ gcc main.o -o main
user@ubuntu:~/test6$ nm main
000000000000063a T add
0000000000201014 B __bss_start
0000000000201014 b completed.7698
                  w __cxa_finalize@@GLIBC_2.2.5
0000000000201000 D __data_start
0000000000201000 W data_start
0000000000000560 t deregister_tm_clones
00000000000005f0 t __do_global_ctors_aux
0000000000200dc0 t __do_global_ctors_aux_fini_array_entry
0000000000201008 D __dso_handle
0000000000200dc8 d _DYNAMIC
0000000000201014 D _edata
0000000000201018 B _end
00000000000006e4 T _fini
0000000000000630 t frame_dummy
0000000000200db8 t __frame_dummy_init_array_entry
000000000000086c r __FRAME_END__
0000000000200fb8 d _GLOBAL_OFFSET_TABLE_
0000000000201010 D globalVar
                  w __gmon_start__
0000000000000704 r __GNU_EH_FRAME_HDR
00000000000004e8 T _init
0000000000200dc0 t __init_array_end
0000000000200db8 t __init_array_start
00000000000006f0 R _IO_stdin_used
                  w _ITM_deregisterTMCloneTable
                  w _ITM_registerTMCloneTable
00000000000006e0 T __libc_csu_fini
0000000000000670 T __libc_csu_init
                  U __libc_start_main@@GLIBC_2.2.5
000000000000064e T main
                  U puts@@GLIBC_2.2.5
00000000000005a0 t register_tm_clones
0000000000000530 T _start
0000000000201018 D __TMC_END__

```

위 사진은 main.o 오브젝트 파일을 main 실행 파일로 변환한 후에 nm 명령어로 어떤 심볼이 있는 지 확인하였다. 위보다 훨씬 많은 심볼이 나타났다. 추가적인 정보로는 w(weak symbol)는 선택적으로 override가 가능하고 확정적이지 않은 심볼로, 링크 시 확정적인 심볼을 만나지 못하면 0으로 설정된다. B, b는 BSS로 초기화되지 않은 전역변수 등으로 심볼이 BSS 섹션에 존재한다. R,

r은 읽기 전용 데이터로, Read-only section에 존재한다.

## 7. objcopy → copy and translate object files

objcopy 명령어는 오브젝트 파일이나 실행 파일 등의 내용을 복사, 추출, 변경 등 하는데 사용되는 GNU binutils에 포함된 도구이다. 이 명령어는 실행 파일을 바이너리로 변환, 특정 섹션 제거, 심볼 제거, 오브젝트 포맷 변경 등 다양한 작업을 할 수 있다.

```
#include <stdio.h>

int main() {
    printf("Hello, objcopy!\n");
    return 0;
}
```

```
user@ubuntu:~/test7$ vi hello.c
user@ubuntu:~/test7$ gcc -o hello hello.c
user@ubuntu:~/test7$ ls
hello hello.c
```

위 사진들은 hello.c라는 c 파일을 작성하고 gcc 명령어를 통해 hello.c 파일로 hello라는 실행 파일을 생성했다. 옵션으로는 S, O, j 등이 있는데 -S는 실행 파일에서 재배치 정보와 심볼을 제거하고, -O는 새로 생성할 파일의 포맷을 지정하고, -j는 실행 파일에서 필요한 섹션만 추출할 수 있다.

```
user@ubuntu:~/test7$ objcopy -O binary hello hello.bin
user@ubuntu:~/test7$ ls
hello hello.bin hello.c
```

위 사진은 실행 파일을 바이너리로 추출한 모습이다. -O binary로 출력 형식을 순수 바이너리로 지정했다.

```
user@ubuntu:~/test7$ objcopy -j .data -O elf64-x86-64 hello.o hello_data.o
user@ubuntu:~/test7$ nm hello_data.o
user@ubuntu:~/test7$ ls
hello hello.bin hello.c hello_data.o hello.o
```

위 사진은 objcopy 명령어를 이용해 data section만 추출해서 새로운 오브젝트 파일을 만들었다. -j .data를 통해 data section만 선택적으로 복사하고 -O elf64-x86-64를 통해 출력 형식을 현재 시스템의 ELF 형식으로 지정했다. 결과적으로 data section만 포함된 hello\_data.o 파일이 만들어졌다. nm hello\_data.o은 hello\_data.o 안의 심볼 목록을 출력하는 것인데 data section에 들어 있는 변수가 존재하지 않아 출력되지 않았다. 마지막으로 ls 명령어를 통해 hello\_data.o 파일이 생성된 것을 확인할 수 있다.

## 8. objdump → display information from object files

objdump는 GNU에서 제공하는 유틸리티로, 오브젝트 파일, 실행 파일, 라이브러리 등의 내부 구조를 분석할 수 있는 도구이다. objdump는 nm보다 훨씬 저수준(어셈블리, 섹션, 헤더, 심볼 등) 정보까지 보여준다.

```
#include <stdio.h>

int globalVar = 10;

void hello() {
    printf("Hello from objdump\n");
}

int main() {
    hello();
    return 0;
}
```

```
user@ubuntu:~/test8$ vi hello.c
user@ubuntu:~/test8$ gcc -o hello hello.c
user@ubuntu:~/test8$ ls
hello  hello.c
```

먼저 간단한 hello.c인 c 파일을 만들고 gcc 컴파일러를 사용해 hello라는 이름의 실행 파일을 만들었다.

```
user@ubuntu:~/test8$ objdump -d hello

hello:      file format elf64-x86-64

Disassembly of section .init:

00000000000004e8 <.init>:
4e8:  48 83 ec 08      sub    $0x8,%rsp
4ec:  48 b5 05 f5 0a 20 00 mov    0x200af5(%rip),%rax        # 200fe8 <__gmon_start__>
4f3:  48 85 c0         test   %rax,%rax
4f6:  74 02          je     4fa<_.init+0x12>
4f8:  ff d0         callq *%rax
4fa:  48 83 c4 08      add    $0x8,%rsp
4fe:  c3             retq

Disassembly of section .plt:

0000000000000500 <.plt>:
500:  ff 35 ba 0a 20 00 pushq  0x200aba(%rip)             # 200fc0 <_GLOBAL_OFFSET_TABLE_+0x8>
506:  ff 25 bc 0a 20 00 jmpq   *0x200abc(%rip)           # 200fc8 <_GLOBAL_OFFSET_TABLE_+0x10>
50c:  0f 1f 40 00      nopl   0x0(%rax)

0000000000000510 <puts@plt>:
510:  ff 25 ba 0a 20 00 jmpq   *0x200aba(%rip)           # 200fd0 <puts@GLIBC_2.2.5>
516:  68 00 00 00 00 pushq  $0x0
51b:  e9 e0 ff ff ff jmpq   500 <.plt>

Disassembly of section .plt.got:

0000000000000520 <__cxa_finalize@plt>:
520:  ff 25 d2 0a 20 00 jmpq   *0x200ad2(%rip)           # 200ff8 <__cxa_finalize@GLIBC_2.2.5>
526:  66 90          xchgb  %ax,%ax

Disassembly of section .text:

0000000000000530 <_start>:
530:  31 ed          xor     %ebp,%ebp
532:  49 89 d1        mov     %rdx,%r9
535:  5e             pop     %rsi
536:  48 89 e2        mov     %rsp,%rdx
539:  48 83 e4 f0     and     $0xfffffffffffffff0,%rsp
53d:  50             push    %rax
53e:  54             push    %rsp
53f:  4c 8d 05 9a 01 00 00 lea     0x19a(%rip),%r8        # 6e0 <__libc_csu_fini>
546:  48 8d 0d 23 01 00 00 lea     0x123(%rip),%rcx       # 670 <__libc_csu_init>
54d:  48 8d 3d f9 00 00 00 lea     0xf9(%rip),%rdi       # 64d <main>
```

위 사진은 objdump 명령어에 -d 옵션을 붙이고 실행 파일 hello를 적었다. 이는 ELF 실행 파일을 디스어셈블(disassemble)하는 명령어이다. 즉, 기계어 → 어셈블리로 번역해서 사람이 읽을 수 있게 보여준다. 주로, text section(코드 영역)을 분석할 때 사용한다.

```

user@ubuntu:~/test8$ objdump -x hello

hello:      file format elf64-x86-64
hello
architecture: i386:x86-64, flags 0x00000150:
HAS_SYMS, DYNAMIC, D_PAGED
start address 0x0000000000000530

Program Header:
  PHDR off   0x0000000000000040 vaddr 0x0000000000000040 paddr 0x0000000000000040 align 2**3
      filesz 0x00000000000001f8 memsz 0x00000000000001f8 flags r--
  INTERP off 0x0000000000000238 vaddr 0x0000000000000238 paddr 0x0000000000000238 align 2**0
      filesz 0x000000000000001c memsz 0x000000000000001c flags r--
  LOAD off   0x0000000000000000 vaddr 0x0000000000000000 paddr 0x0000000000000000 align 2**21
      filesz 0x0000000000000878 memsz 0x0000000000000878 flags r-x
  LOAD off   0x0000000000000db8 vaddr 0x0000000000200db8 paddr 0x0000000000200db8 align 2**21
      filesz 0x000000000000025c memsz 0x0000000000000260 flags rw-
  DYNAMIC off 0x0000000000000dc8 vaddr 0x0000000000200dc8 paddr 0x0000000000200dc8 align 2**3
      filesz 0x00000000000001f0 memsz 0x00000000000001f0 flags rw-
  NOTE off   0x0000000000000254 vaddr 0x0000000000000254 paddr 0x0000000000000254 align 2**2
      filesz 0x0000000000000044 memsz 0x0000000000000044 flags r--
EH_FRAME off 0x0000000000000708 vaddr 0x0000000000000708 paddr 0x0000000000000708 align 2**2
      filesz 0x0000000000000044 memsz 0x0000000000000044 flags r--
  STACK off   0x0000000000000000 vaddr 0x0000000000000000 paddr 0x0000000000000000 align 2**4
      filesz 0x0000000000000000 memsz 0x0000000000000000 flags rw-
  RELRO off   0x0000000000000db8 vaddr 0x0000000000200db8 paddr 0x0000000000200db8 align 2**0
      filesz 0x0000000000000248 memsz 0x0000000000000248 flags r--

Dynamic Section:
NEEDED      libc.so.6
INIT        0x00000000000004e8
FINI        0x00000000000006e4
INIT_ARRAY  0x0000000000200db8
INIT_ARRAYSZ 0x0000000000000008
FINI_ARRAY  0x0000000000200dc0
FINI_ARRAYSZ 0x0000000000000008
GNU_HASH    0x0000000000000298
STRTAB      0x0000000000000360
SYMTAB      0x00000000000002b8
STRSZ       0x0000000000000082
SYMENT      0x0000000000000018
DEBUG       0x0000000000000000
PLTGOT      0x0000000000200fb8
PLTRELSZ    0x0000000000000018
PLTREL      0x0000000000000007
JMPREL      0x00000000000004d0
RELA        0x0000000000000410
RELASZ      0x00000000000000c0
RELAENT     0x0000000000000018

```

위 사진은 objdump 명령어에 -x 옵션을 붙이고 실행 파일 hello를 적었다. 이걸 objdump에서 파일의 전체 구조, 헤더, 동적 정보까지 다 보여주는 명령어이다. 즉, 실행 파일 또는 오브젝트 파일의 전체 헤더 정보(내부 구조)를 출력해준다.

```

user@ubuntu:~/test8$ objdump -h hello

hello:      file format elf64-x86-64

Sections:
Idx Name          Size      VMA           LMA           File off  Algn
  0 .interp        0000001c  000000000000238 000000000000238 00000238  2**0
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .note.ABI-tag  00000020  000000000000254 000000000000254 00000254  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  2 .note.gnu.build-id 00000024  000000000000274 000000000000274 00000274  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  3 .gnu.hash       0000001c  000000000000298 000000000000298 00000298  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  4 .dynsym         000000a8  0000000000002b8 0000000000002b8 000002b8  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  5 .dynstr         00000082  000000000000360 000000000000360 00000360  2**0
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 .gnu.version    0000000e  0000000000003e2 0000000000003e2 000003e2  2**1
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  7 .gnu.version_r  00000020  0000000000003f0 0000000000003f0 000003f0  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  8 .rela.dyn       000000c0  000000000000410 000000000000410 00000410  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  9 .rela.plt       00000018  0000000000004d0 0000000000004d0 000004d0  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
 10 .init           00000017  0000000000004e8 0000000000004e8 000004e8  2**2
    CONTENTS, ALLOC, LOAD, READONLY, CODE
 11 .plt            00000020  000000000000500 000000000000500 00000500  2**4
    CONTENTS, ALLOC, LOAD, READONLY, CODE
 12 .plt.got        00000008  000000000000520 000000000000520 00000520  2**3
    CONTENTS, ALLOC, LOAD, READONLY, CODE
 13 .text           000001b2  000000000000530 000000000000530 00000530  2**4
    CONTENTS, ALLOC, LOAD, READONLY, CODE
 14 .fini           00000009  0000000000006e4 0000000000006e4 000006e4  2**2
    CONTENTS, ALLOC, LOAD, READONLY, CODE
 15 .rodata         00000017  0000000000006f0 0000000000006f0 000006f0  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
 16 .eh_frame_hdr   00000044  000000000000708 000000000000708 00000708  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
 17 .eh_frame       00000128  000000000000750 000000000000750 00000750  2**3
    CONTENTS, ALLOC, LOAD, READONLY, DATA
 18 .init_array     00000008  0000000000200db8 0000000000200db8 00000db8  2**3
    CONTENTS, ALLOC, LOAD, DATA
 19 .fini_array     00000008  0000000000200dc0 0000000000200dc0 00000dc0  2**3
    CONTENTS, ALLOC, LOAD, DATA
 20 .dynamic        000001f0  0000000000200dc8 0000000000200dc8 00000dc8  2**3
    CONTENTS, ALLOC, LOAD, DATA
 21 .got            00000048  0000000000200fb8 0000000000200fb8 00000fb8  2**3
    CONTENTS, ALLOC, LOAD, DATA

```

위 사진은 objdump 명령어에 -h 옵션을 붙이고 실행 파일 hello를 적었다. 이 명령은 실행 파일의 섹션 헤더 정보를 출력하는 명령이다. 즉, ELF 파일 안에 들어 있는 모든 섹션(text, data 등)의 위치, 크기, 속성 등을 보여준다.

```

user@ubuntu:~/test8$ objdump -S hello

hello:      file format elf64-x86-64

Disassembly of section .init:

00000000000004e8 <_init>:
4e8:  48 83 ec 08          sub    $0x8,%rsp
4ec:  48 8b 05 f5 0a 20 00  mov    0x200af5(%rip),%rax      # 200fe8 <__gmon_start__>
4f3:  48 85 c0             test   %rax,%rax
4f6:  74 02              je     4fa <_init+0x12>
4f8:  ff d0             callq  *%rax
4fa:  48 83 c4 08          add    $0x8,%rsp
4fe:  c3                retq

Disassembly of section .plt:

0000000000000500 <.plt>:
500:  ff 35 ba 0a 20 00    pushq 0x200aba(%rip)          # 200fc0 <_GLOBAL_OFFSET_TABLE_+0x8>
506:  ff 25 bc 0a 20 00    jmpq   *0x200abc(%rip)        # 200fc8 <_GLOBAL_OFFSET_TABLE_+0x10>
50c:  0f 1f 40 00          nopl   0x0(%rax)

0000000000000510 <puts@plt>:
510:  ff 25 ba 0a 20 00    jmpq   *0x200aba(%rip)        # 200fd0 <puts@GLIBC_2.2.5>
516:  68 00 00 00 00      pushq  $0x0
51b:  e9 e0 ff ff ff      jmpq   500 <.plt>

Disassembly of section .plt.got:

0000000000000520 <__cxa_finalize@plt>:
520:  ff 25 d2 0a 20 00    jmpq   *0x200ad2(%rip)        # 200ff8 <__cxa_finalize@GLIBC_2.2.5>
526:  66 90              xchgb  %ax,%ax

Disassembly of section .text:

0000000000000530 <_start>:
530:  31 ed              xor     %ebp,%ebp
532:  49 89 d1           mov     %rdx,%r9
535:  5e                pop     %rsi
536:  48 89 e2           mov     %rsp,%rdx
539:  48 83 e4 f0        and     $0xfffffffffffffff0,%rsp
53d:  50                push    %rax
53e:  54                push    %rsp
53f:  4c 8d 05 9a 01 00 00 lea     0x19a(%rip),%r8        # 6e0 <__libc_csu_fini>
546:  48 8d 0d 23 01 00 00 lea     0x123(%rip),%rcx       # 670 <__libc_csu_init>
54d:  48 8d 3d f9 00 00 00 lea     0xf9(%rip),%rdi        # 64d <main>
554:  ff 15 86 0a 20 00    callq  *0x200a86(%rip)        # 200fe0 <__libc_start_main@GLIBC_2.2.5>
55a:  f4                hlt
55b:  0f 1f 44 00 00      nopl   0x0(%rax,%rax,1)

```

위 사진은 objdump 명령어에 -S 옵션을 붙이고 실행 파일 hello를 적었다. 이 명령은 objdump -d처럼 디스어셈블(기계어 → 어셈블리)을 하면서, 가능하면 원래의 C 소스 코드도 함께 출력해주는 명령어이다. 즉, C 코드와 어셈블리 코드가 나란히 나오는 고급 분석 뷰라고 할 수 있다. 하지만 gcc -g로 디버그 심볼이 포함되어 있어야 -S가 C 소스 코드를 보여줄 수 있다.

```

user@ubuntu:~/test8$ objdump -s hello

hello:      file format elf64-x86-64

Contents of section .interp:
0238 2f6c6962 36342f6c 642d6c69 6e75782d /lib64/ld-linux-
0248 7838362d 36342e73 6f2e3200 x86-64.so.2.
Contents of section .note.ABI-tag:
0254 04000000 10000000 01000000 474e5500 .....GNU.
0264 00000000 03000000 02000000 00000000 .....
Contents of section .note.gnu.build-id:
0274 04000000 14000000 03000000 474e5500 .....GNU.
0284 8c8817c2 9536f4da 87102722 61ab08aa .....6....'"a...
0294 7c175975 |.Yu
Contents of section .gnu.hash:
0298 01000000 01000000 01000000 00000000 .....
02a8 00000000 00000000 00000000 .....
Contents of section .dynsym:
02b8 00000000 00000000 00000000 00000000 .....
02c8 00000000 00000000 3d000000 20000000 .....=.
02d8 00000000 00000000 00000000 00000000 .....
02e8 0b000000 12000000 00000000 00000000 .....
02f8 00000000 00000000 1f000000 12000000 .....
0308 00000000 00000000 00000000 00000000 .....
0318 59000000 20000000 00000000 00000000 Y...
0328 00000000 00000000 68000000 20000000 .....h...
0338 00000000 00000000 00000000 00000000 .....
0348 10000000 22000000 00000000 00000000 ....".....
0358 00000000 00000000 .....
Contents of section .dynstr:
0360 006c6962 632e736f 2e360070 75747300 .libc.so.6.puts.
0370 5f5f6378 615f6669 6e616c69 7a65005f __cxa_finalize._
0380 5f6c6962 635f7374 6172745f 6d61696e _libc_start_main
0390 00474c49 42435f32 2e322e35 005f4954 .GLIBC_2.2.5._IT
03a0 4d5f6465 72656769 73746572 544d436c M_deregisterTMCl
03b0 6f6e6554 61626c65 005f5f67 6d6f6e5f oneTable.__gmon_
03c0 73746172 745f5f00 5f49544d 5f726567 start__._ITM_reg
03d0 69737465 72544d43 6c6f6e65 5461626c isterTMCloneTabl
03e0 6500 e.
Contents of section .gnu.version:
03e2 00000000 02000200 00000000 0200 .....
Contents of section .gnu.version_r:
03f0 01000100 01000000 10000000 00000000 .....
0400 751a6909 00000200 31000000 00000000 u.i.....1.....
Contents of section .rela.dyn:
0410 b80d2000 00000000 08000000 00000000 ..
0420 30060000 00000000 c00d2000 00000000 0.....
0430 08000000 00000000 f0050000 00000000 .....
0440 08102000 00000000 08000000 00000000 ..
0450 08102000 00000000 d80f2000 00000000 ..

```

위 사진은 objdump 명령어에 -s 옵션을 붙이고 실행 파일 hello를 적었다. 이 명령은 hello 실행 파일의 섹션 내용을 바이트 단위로 dump해준다. 즉, 파일 내부 내용을 16진수와 ASCII로 보여주는 바이너리 뷰라고 생각하며 된다.

```

user@ubuntu:~/test8$ objdump -t hello

hello:      file format elf64-x86-64

SYMBOL TABLE:
0000000000000238 l d .interp 0000000000000000 .interp
0000000000000254 l d .note.ABI-tag 0000000000000000 .note.ABI-tag
0000000000000274 l d .note.gnu.build-id 0000000000000000 .note.gnu.build-id
0000000000000298 l d .gnu.hash 0000000000000000 .gnu.hash
00000000000002b8 l d .dynsym 0000000000000000 .dynsym
0000000000000360 l d .dynstr 0000000000000000 .dynstr
00000000000003e2 l d .gnu.version 0000000000000000 .gnu.version
00000000000003f0 l d .gnu.version_r 0000000000000000 .gnu.version_r
0000000000000410 l d .rela.dyn 0000000000000000 .rela.dyn
00000000000004d0 l d .rela.plt 0000000000000000 .rela.plt
00000000000004e8 l d .init 0000000000000000 .init
0000000000000500 l d .plt 0000000000000000 .plt
0000000000000520 l d .plt.got 0000000000000000 .plt.got
0000000000000530 l d .text 0000000000000000 .text
00000000000006e4 l d .fini 0000000000000000 .fini
00000000000006f0 l d .rodata 0000000000000000 .rodata
0000000000000708 l d .eh_frame_hdr 0000000000000000 .eh_frame_hdr
0000000000000750 l d .eh_frame 0000000000000000 .eh_frame
00000000000020db8 l d .init_array 0000000000000000 .init_array
00000000000020dc0 l d .fini_array 0000000000000000 .fini_array
00000000000020dc8 l d .dynamic 0000000000000000 .dynamic
00000000000020fb8 l d .got 0000000000000000 .got
00000000000020100 l d .data 0000000000000000 .data
00000000000020104 l d .bss 0000000000000000 .bss
0000000000000000 l d .comment 0000000000000000 .comment
0000000000000000 l df *ABS* 0000000000000000 crtstuff.c
0000000000000560 l F .text 0000000000000000 deregister_tm_clones
00000000000005a0 l F .text 0000000000000000 register_tm_clones
00000000000005f0 l F .text 0000000000000000 __do_global_ctors_aux
00000000000020104 l O .bss 0000000000000001 completed.7698
00000000000020dc0 l O .fini_array 0000000000000000 __do_global_ctors_aux_fini_array_entry
0000000000000630 l F .text 0000000000000000 frame_dummy
00000000000020db8 l O .init_array 0000000000000000 __frame_dummy_init_array_entry
0000000000000000 l df *ABS* 0000000000000000 hello.c
0000000000000000 l df *ABS* 0000000000000000 crtstuff.c
0000000000000874 l O .eh_frame 0000000000000000 __FRAME_END__
0000000000000000 l df *ABS* 0000000000000000
00000000000020dc0 l .init_array 0000000000000000 __init_array_end
00000000000020dc8 l O .dynamic 0000000000000000 DYNAMIC
00000000000020db8 l .init_array 0000000000000000 __init_array_start
0000000000000708 l .eh_frame_hdr 0000000000000000 GNU_EH_FRAME_HDR
00000000000020fb8 l O .got 0000000000000000 _GLOBAL_OFFSET_TABLE_
00000000000006e0 g F .text 0000000000000002 __libc_csu_fini
0000000000000000 w *UND* 0000000000000000 __ITM_deregisterTMCloneTable
00000000000020100 w .data 0000000000000000 data_start
0000000000000000 F *UND* 0000000000000000 puts@@GLIBC_2.2.5

```

위 사진은 objdump 명령어에 -t 옵션을 붙이고 실행 파일 hello를 적었다. 이 명령은 ELF 실행 파일 hello의 심볼 테이블을 출력해주는 명령이다. 이걸 nm hello와 비슷하지만, objdump는 좀 더 포맷이 정제되어 있고, 다른 출력들과 통합해서 보기 좋게 만들 수 있다.

```

user@ubuntu:~/test8$ objdump -f hello

hello:      file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000150:
HAS_SYMS, DYNAMIC, D_PAGED
start address 0x00000000000000530

```

위 사진은 objdump 명령어에 -f 옵션을 붙이고 실행 파일 hello를 적었다. 이 명령은 ELF 실행 파일의 파일 헤더 정보만 간단하게 요약해서 보여주는 명령이다.

## 9. readelf → Displays information about ELF files

소스코드를 컴파일하면 목적코드로 변환되고, 목적코드와 필요한 라이브러리들을 링커 커맨드 파일 정보를 통해 Linking하는 과정이 끝나면 실행가능한 이진형식 파일인 Executable and Linkable Format 파일이 생성된다. readelf 명령어는 ELF 파일을 분석하기 위한 전용 도구로 objdump보다 ELF 헤더 구조 중심에 초점을 둔다. 이 명령어는 오브젝트 파일(.o), 실행 파일, 라이브러리 모두 분석이 가능하다.

```
user@ubuntu:~/test$ readelf -h hello
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                               UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Shared object file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x530
  Start of program headers:              64 (bytes into file)
  Start of section headers:             8720 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:              9
  Size of section headers:               64 (bytes)
  Number of section headers:             34
  Section header string table index:     33
```

위 사진은 readelf 명령어에 -h 옵션을 붙인 것으로, ELF 헤더는 ELF 파일의 시작 부분에 위치하여 ELF 파일의 형식, 아키텍처, 섹션 및 프로그램 헤더의 위치 및 크기 등을 정의한다.

```

user@ubuntu:~/test9$ readelf -S hello
There are 34 section headers, starting at offset 0x2210:

Section Headers:
[Nr] Name              Type              Address            Offset
     Size              EntSize          Flags Link Info Align
[ 0]                      NULL              0000000000000000  00000000
     0000000000000000  0000000000000000  0 0 0
[ 1] .interp              PROGBITS          0000000000000238  00000238
     000000000000001c  0000000000000000  A 0 0 1
[ 2] .note.ABI-tag        NOTE              0000000000000254  00000254
     0000000000000020  0000000000000000  A 0 0 4
[ 3] .note.gnu.build-id   NOTE              0000000000000274  00000274
     0000000000000024  0000000000000000  A 0 0 4
[ 4] .gnu.hash            GNU_HASH          0000000000000298  00000298
     000000000000001c  0000000000000000  A 5 0 8
[ 5] .dynsym              DYNSYM            00000000000002b8  000002b8
     00000000000000a8  0000000000000018  A 6 1 8
[ 6] .dynstr              STRTAB            0000000000000360  00000360
     0000000000000082  0000000000000000  A 0 0 1
[ 7] .gnu.version          VERSYM            00000000000003e2  000003e2
     000000000000000e  0000000000000002  A 5 0 2
[ 8] .gnu.version_r        VERNEED           00000000000003f0  000003f0
     0000000000000020  0000000000000000  A 6 1 8
[ 9] .rela.dyn             RELA              0000000000000410  00000410
     00000000000000c0  0000000000000018  A 5 0 8
[10] .rela.plt             RELA              00000000000004d0  000004d0
     0000000000000018  0000000000000018  AI 5 22 8
[11] .init                PROGBITS          00000000000004e8  000004e8
     0000000000000017  0000000000000000  AX 0 0 4
[12] .plt                 PROGBITS          0000000000000500  00000500
     0000000000000020  0000000000000010  AX 0 0 16
[13] .plt.got             PROGBITS          0000000000000520  00000520
     0000000000000008  0000000000000008  AX 0 0 8
[14] .text                PROGBITS          0000000000000530  00000530
     00000000000001b2  0000000000000000  AX 0 0 16
[15] .fini                PROGBITS          00000000000006e4  000006e4
     0000000000000009  0000000000000000  AX 0 0 4
[16] .rodata              PROGBITS          00000000000006f0  000006f0
     0000000000000017  0000000000000000  A 0 0 4
[17] .eh_frame_hdr        PROGBITS          0000000000000708  00000708
     0000000000000044  0000000000000000  A 0 0 4
[18] .eh_frame            PROGBITS          0000000000000750  00000750
     0000000000000128  0000000000000000  A 0 0 8
[19] .init_array           INIT_ARRAY         0000000000200db8  00000db8
     0000000000000008  0000000000000008  WA 0 0 8
[20] .fini_array           FINI_ARRAY         0000000000200dc0  00000dc0
     0000000000000008  0000000000000008  WA 0 0 8
[21] .dynamic              DYNAMIC           0000000000200dc8  00000dc8
     0000000000000150  0000000000000010  WA 6 0 8

```

위 사진은 readelf 명령어에 -S 옵션을 붙인 것으로, 섹션 헤더 테이블은 섹션 헤더들의 배열로 구성되어 있으며, 각 섹션 헤더는 섹션의 속성과 위치 등을 정의한다. 섹션은 프로그램의 코드, 데이터, 심볼 테이블, 디버깅 정보 등이 포함된다.

```

user@ubuntu:~/test9$ readelf -l hello

Elf file type is DYN (Shared object file)
Entry point 0x530
There are 9 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
     FileSiz      MemSiz          Flags  Align
PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
              0x00000000000001f8 0x00000000000001f8  R      0x8
INTERP         0x0000000000000238 0x0000000000000238 0x0000000000000238
              0x000000000000001c 0x000000000000001c  R      0x1
    [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
              0x0000000000000878 0x0000000000000878  R E    0x200000
LOAD           0x0000000000000db8 0x0000000000200db8 0x0000000000200db8
              0x000000000000025c 0x0000000000000260  RW     0x200000
DYNAMIC        0x0000000000000dc8 0x0000000000200dc8 0x0000000000200dc8
              0x00000000000001f0 0x00000000000001f0  RW     0x8
NOTE           0x0000000000000254 0x0000000000000254 0x0000000000000254
              0x0000000000000044 0x0000000000000044  R      0x4
GNU_EH_FRAME   0x0000000000000708 0x0000000000000708 0x0000000000000708
              0x0000000000000044 0x0000000000000044  R      0x4
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
              0x0000000000000000 0x0000000000000000  RW    0x10
GNU_RELRO      0x0000000000000db8 0x0000000000200db8 0x0000000000200db8
              0x0000000000000248 0x0000000000000248  R      0x1

Section to Segment mapping:
Segment Sections...
00
01      .interp
02      .interp .note.ABI-tag .note.gnu.build-id .gnu.hash .dynsym .dynstr .gnu.version .gnu.version_r .rela.dyn .rel
a.plt .init .plt .plt.got .text .fini .rodata .eh_frame_hdr .eh_frame
03      .init_array .fini_array .dynamic .got .data .bss
04      .dynamic
05      .note.ABI-tag .note.gnu.build-id
06      .eh_frame_hdr
07
08      .init_array .fini_array .dynamic .got

```

위 사진은 readelf 명령어에 -l 옵션을 붙인 것으로, 프로그램 헤더 테이블은 프로세스가 실행될 때 메모리에 로드되는 세그먼트의 속성과 위치를 지정하는 세그먼트 헤더 엔트리들로 구성되어 있다. 따라서 실행 파일을 실행할 때 운영체제가 이 정보를 사용하여 프로그램을 적절한 메모리 위치에 로드하게 된다.

```

user@ubuntu:~/test9$ readelf -s hello

Symbol table '.dynsym' contains 7 entries:
   Num:    Value          Size Type      Bind     Vis     Ndx Name
   0: 0000000000000000      0 NOTYPE   LOCAL   DEFAULT   UND
   1: 0000000000000000      0 NOTYPE   WEAK    DEFAULT   UND __ITM_deregisterTMCloneTab
   2: 0000000000000000      0 FUNC     GLOBAL   DEFAULT   UND puts@GLIBC_2.2.5 (2)
   3: 0000000000000000      0 FUNC     GLOBAL   DEFAULT   UND __libc_start_main@GLIBC_2.2.5 (2)
   4: 0000000000000000      0 NOTYPE   WEAK    DEFAULT   UND __gmon_start__
   5: 0000000000000000      0 NOTYPE   WEAK    DEFAULT   UND __ITM_registerTMCloneTable
   6: 0000000000000000      0 FUNC     WEAK    DEFAULT   UND __cxa_finalize@GLIBC_2.2.5 (2)

Symbol table '.symtab' contains 70 entries:
   Num:    Value          Size Type      Bind     Vis     Ndx Name
   0: 0000000000000000      0 NOTYPE   LOCAL   DEFAULT   UND
   1: 00000000000000238      0 SECTION  LOCAL   DEFAULT     1
   2: 00000000000000254      0 SECTION  LOCAL   DEFAULT     2
   3: 00000000000000274      0 SECTION  LOCAL   DEFAULT     3
   4: 00000000000000298      0 SECTION  LOCAL   DEFAULT     4
   5: 000000000000002b8      0 SECTION  LOCAL   DEFAULT     5
   6: 00000000000000360      0 SECTION  LOCAL   DEFAULT     6
   7: 000000000000003e2      0 SECTION  LOCAL   DEFAULT     7
   8: 000000000000003f0      0 SECTION  LOCAL   DEFAULT     8
   9: 00000000000000410      0 SECTION  LOCAL   DEFAULT     9
  10: 000000000000004d0      0 SECTION  LOCAL   DEFAULT    10
  11: 000000000000004e8      0 SECTION  LOCAL   DEFAULT    11
  12: 00000000000000500      0 SECTION  LOCAL   DEFAULT    12
  13: 00000000000000520      0 SECTION  LOCAL   DEFAULT    13
  14: 00000000000000530      0 SECTION  LOCAL   DEFAULT    14
  15: 000000000000006e4      0 SECTION  LOCAL   DEFAULT    15
  16: 000000000000006f0      0 SECTION  LOCAL   DEFAULT    16
  17: 00000000000000708      0 SECTION  LOCAL   DEFAULT    17
  18: 00000000000000750      0 SECTION  LOCAL   DEFAULT    18
  19: 0000000000200db8      0 SECTION  LOCAL   DEFAULT    19
  20: 0000000000200dc0      0 SECTION  LOCAL   DEFAULT    20
  21: 0000000000200dc8      0 SECTION  LOCAL   DEFAULT    21
  22: 0000000000200fb8      0 SECTION  LOCAL   DEFAULT    22
  23: 0000000000201000      0 SECTION  LOCAL   DEFAULT    23
  24: 0000000000201014      0 SECTION  LOCAL   DEFAULT    24
  25: 0000000000000000      0 SECTION  LOCAL   DEFAULT    25
  26: 0000000000000000      0 SECTION  LOCAL   DEFAULT    26
  27: 0000000000000000      0 SECTION  LOCAL   DEFAULT    27
  28: 0000000000000000      0 SECTION  LOCAL   DEFAULT    28
  29: 0000000000000000      0 SECTION  LOCAL   DEFAULT    29
  30: 0000000000000000      0 SECTION  LOCAL   DEFAULT    30
  31: 0000000000000000      0 FILE      LOCAL   DEFAULT   ABS crtstuff.c
  32: 00000000000000560      0 FUNC     LOCAL   DEFAULT    14 deregister_tm_clones
  33: 000000000000005a0      0 FUNC     LOCAL   DEFAULT    14 register_tm_clones
  34: 000000000000005f0      0 FUNC     LOCAL   DEFAULT    14 __do_global_ctors_aux
  35: 0000000000201014      1 0035CT  LOCAL   DEFAULT    24 __cxa_finalize@GLIBC_2.2.5

```

위 사진은 readelf 명령어에 -s 옵션을 붙인 것으로, 심볼 테이블은 섹션 중 하나로, 프로그램의 변수, 함수, 외부 참조 등에 대한 정보를 담고 있다. 심볼 테이블은 디버깅이나 동적 링킹 등에서 사용된다.

```

user@ubuntu:~/test9$ readelf -r hello

Relocation section '.rela.dyn' at offset 0x410 contains 8 entries:
   Offset             Info                Type             Sym. Value      Sym. Name + Addend
000000200db8 0000000000000008 R_X86_64_RELATIVE             630
000000200dc0 0000000000000008 R_X86_64_RELATIVE             5f0
000000201008 0000000000000008 R_X86_64_RELATIVE             201008
000000200fd8 0001000000000006 R_X86_64_GLOB_DAT 0000000000000000 __ITM_deregisterTMClone + 0
000000200fe0 0003000000000006 R_X86_64_GLOB_DAT 0000000000000000 __libc_start_main@GLIBC_2.2.5 + 0
000000200fe8 0004000000000006 R_X86_64_GLOB_DAT 0000000000000000 __gmon_start__ + 0
000000200ff0 0005000000000006 R_X86_64_GLOB_DAT 0000000000000000 __ITM_registerTMCloneTa + 0
000000200ff8 0006000000000006 R_X86_64_GLOB_DAT 0000000000000000 __cxa_finalize@GLIBC_2.2.5 + 0

Relocation section '.rela.plt' at offset 0x4d0 contains 1 entry:
   Offset             Info                Type             Sym. Value      Sym. Name + Addend
000000200fd0 0002000000000007 R_X86_64_JUMP_SLO 0000000000000000 puts@GLIBC_2.2.5 + 0

```

위 사진은 readelf 명령어에 -r 옵션을 붙인 것으로, 재배포치 정보를 보여주고, 동적 링커가 사용해야 할 재배포치 항목 정보를 보여준다.

```

user@ubuntu:~/test9$ readelf -d hello

Dynamic section at offset 0xdc8 contains 27 entries:
   Tag               Type                             Name/Value
0x0000000000000001 (NEEDED)                             Shared library: [libc.so.6]
0x000000000000000c (INIT)                               0x4e8
0x000000000000000d (FINI)                               0x6e4
0x0000000000000019 (INIT_ARRAY)                            0x200db8
0x000000000000001b (INIT_ARRAYSZ)                        8 (bytes)
0x000000000000001a (FINI_ARRAY)                            0x200dc0
0x000000000000001c (FINI_ARRAYSZ)                        8 (bytes)
0x000000006ffffff5 (GNU_HASH)                           0x298
0x0000000000000005 (STRTAB)                             0x360
0x0000000000000006 (SYMTAB)                             0x2b8
0x000000000000000a (STRSZ)                               130 (bytes)
0x000000000000000b (SYMENT)                             24 (bytes)
0x0000000000000015 (DEBUG)                               0x0
0x0000000000000003 (PLTGOT)                             0x200fb8
0x0000000000000002 (PLTRELSZ)                           24 (bytes)
0x0000000000000014 (PLTREL)                             RELA
0x0000000000000017 (JMPREL)                             0x4d0
0x0000000000000007 (RELA)                               0x410
0x0000000000000008 (RELASZ)                             192 (bytes)
0x0000000000000009 (RELAENT)                            24 (bytes)
0x000000000000001e (FLAGS)                             BIND_NOW
0x000000006fffffff (FLAGS_1)                            Flags: NOW PIE
0x000000006ffffffe (VERNEED)                            0x3f0
0x000000006fffffff (VERNEEDNUM)                         1
0x000000006ffffff0 (VERSYM)                             0x3e2
0x000000006ffffff9 (RELACOUNT)                          3
0x0000000000000000 (NULL)                               0x0

```

위 사진은 readelf 명령어에 -d 옵션을 붙인 것으로, 동적 섹션을 보여주고, 동적 링크 정보를 출력해준다(필요한 라이브러리 등).

```

user@ubuntu:~/test9$ readelf -p .strtab hello

String dump of section '.strtab':
 [ 1] crtstuff.c
 [ c] deregister_tm_clones
 [21] __do_global_ctors_aux
 [37] completed.7698
 [46] __do_global_ctors_aux_fini_array_entry
 [6d] frame_dummy
 [79] __frame_dummy_init_array_entry
 [98] hello.c
 [a0] __FRAME_END__
 [ae] __init_array_end
 [bf] __DYNAMIC
 [c8] __init_array_start
 [db] __GNU_EH_FRAME_HDR
 [ee] __GLOBAL_OFFSET_TABLE__
[104] __libc_csu_fini
[114] __ITM_deregisterTMCloneTable
[130] puts@@GLIBC_2.2.5
[142] __edata
[149] globalVar
[153] __libc_start_main@@GLIBC_2.2.5
[172] hello
[178] __data_start
[185] __gmon_start__
[194] __dso_handle
[1a1] __IO_stdin_used
[1b0] __libc_csu_init
[1c0] __bss_start
[1cc] main
[1d1] __TMC_END__
[1dd] __ITM_registerTMCloneTable
[1f7] __cxa_finalize@@GLIBC_2.2.5

```

위 사진은 readelf 명령어에 -p 옵션을 붙인 것으로, 문자열 테이블을 보여주고, .strtab(또

는 .dynstr)에 저장된 모든 심볼 이름 문자열을 출력해준다.

```
user@ubuntu:~/test9$ readelf -x .strtab hello
Hex dump of section '.strtab':
0x00000000 00637274 73747566 662e6300 64657265 .crtstuff.c.dere
0x00000010 67697374 65725f74 6d5f636c 6f6e6573 gister_tm_clones
0x00000020 005f5f64 6f5f676c 6f62616c 5f64746f .__do_global_eto
0x00000030 72735f61 75780063 6f6d706c 65746564 rs_aux.completed
0x00000040 2e373639 38005f5f 646f5f67 6c6f6261 .7698. __do_globa
0x00000050 6c5f6474 6f72735f 6175785f 66696e69 l_dtors_aux_fini
0x00000060 5f617272 61795f65 6e747279 00667261 _array_entry.fra
0x00000070 6d655f64 756d6d79 005f5f66 72616d65 me_dummy.__frame
0x00000080 5f64756d 6d795f69 6e69745f 61727261 _dummy_init arra
0x00000090 795f656e 74727900 68656c6c 6f2e6300 y_entry.hello.c.
0x000000a0 5f5f4652 414d455f 454e445f 5f005f5f __FRAME_END__
0x000000b0 696e6974 5f617272 61795f65 6e64005f init_array_end.
0x000000c0 44594e41 4d494300 5f5f696e 69745f61 DYNAMIC.__init_a
0x000000d0 72726179 5f737461 7274005f 5f474e55 rray_start.__GNU
0x000000e0 5f45485f 4652414d 455f4844 52005f47 _EH_FRAME HDR. G
0x000000f0 4c4f4241 4c5f4f46 46534554 5f544142 GLOBAL_OFFSET_TAB
0x00000100 4c455f00 5f5f6c69 62635f63 73755f66 LE.__libc_csu_f
0x00000110 696e6900 5f49544d 5f646572 65676973 ini_ITM_deregis
0x00000120 74657254 4d436c6f 6e655461 626c6500 terTMCCloneTable.
0x00000130 70757473 4040474c 4942435f 322e322e puts@@GLIBC_2.2.
0x00000140 35005f65 64617461 00676c6f 62616c56 5_edata.globalV
0x00000150 6172005f 5f6c6962 635f7374 6172745f ar.__libc_start_
0x00000160 6d61696e 4040474c 4942435f 322e322e main@@GLIBC_2.2.
0x00000170 35006865 6c6c6f00 5f5f6461 74615f73 5.hello.__data_s
0x00000180 74617274 005f5f67 6d6f6e5f 73746172 tart.__gmon_star
0x00000190 745f5f00 5f5f6473 6f5f6861 6e646c65 t__dso_handle
0x000001a0 005f494f 5f737464 696e5f75 73656400 _IO_stdin_used.
0x000001b0 5f5f6c69 62635f63 73755f69 6e697400 __libc_csu_init.
0x000001c0 5f5f6273 735f7374 61727400 6d61696e __bss_start.main
0x000001d0 005f5f54 4d435f45 4e445f5f 005f4954 .TMC_END__IT
0x000001e0 4d5f7265 67697374 6572544d 436c6f6e M_registerTMClo
0x000001f0 65546162 6c65005f 5f637861 5f66696e eTable.__cxa_fln
0x00000200 616c697a 65404047 4c494243 5f322e32 alize@@GLIBC_2.2
0x00000210 2e3500 .5.
```

위 사진은 readelf 명령어에 -x 옵션을 붙인 것으로, 스트링 테이블은 섹션 중 하나로, 다른 섹션 및 테이블에서 사용되는 문자열 데이터를 null-terminated 문자열로 저장한다. 심볼 테이블에서 st\_name(문자열 테이블에서 심볼 이름의 오프셋)으로 각 심볼의 이름을 참조할 때 사용되며, 디버깅 정보나 다른 섹션에서 사용되는 문자열 데이터에 대한 인덱스를 제공한다.

## 10. size → list section sizes and total size

size 명령어는 ELF 파일 또는 .o 파일에 대해 코드(.text), 초기화된 데이터(.data), 초기화되지 않은 데이터(.bss), 그리고 전체 합계를 보여준다. 이는 메모리 사용량 분석, 펌웨어 사이즈 체크, 최적화 전후 비교 등에 이용된다.

```
user@ubuntu:~/test10$ ls -alh hello
-rwxrwxr-x 1 user user 8.2K Apr 10 13:59 hello
user@ubuntu:~/test10$ size hello
text    data    bss     dec     hex filename
1519    604     12     2135    857 hello
```

size 명령어는 오브젝트 파일이나 일반 파일에서 섹션이나 파일의 전체 크기를 출력한다. 먼저 위와 같이 ls 명령으로 라이브러리의 전체 크기를 볼 수 있다. 그리고 그 아래에 size 명령은 각 섹

션에 대한 정보를 출력한다.

```
user@ubuntu:~/test10$ size -t hello
text    data    bss     dec     hex filename
1519    604     12      2135    857 hello
1519    604     12      2135    857 (TOTALS)
```

위 사진과 같이 size 명령어에 -t 옵션은 지정한 파일의 전체 합계를 출력해준다.

```
user@ubuntu:~/test10$ size --format=berkeley hello
text    data    bss     dec     hex filename
1519    604     12      2135    857 hello
user@ubuntu:~/test10$ size --format=sysv hello
hello :
section      size      addr
.interp      28        568
.note.ABI-tag 32        596
.note.gnu.build-id 36        628
.gnu.hash    28        664
.dynsym      168       696
.dynstr      130       864
.gnu.version  14        994
.gnu.version_r 32       1008
.rela.dyn    192       1040
.rela.plt    24        1232
.init        23        1256
.plt         32        1280
.plt.got     8         1312
.text        418       1328
.fini        9         1748
.rodata      21        1760
.eh_frame_hdr 60        1784
.eh_frame    264       1848
.init_array  8         2100664
.fini_array  8         2100672
.dynamic     496       2100680
.got         72        2101176
.data        20        2101248
.bss         12        2101268
.comment     41        0
Total       2176
```

위 사진과 같이 size의 출력 형태에는 버클리 와 System V가 있다. 출력 형태는 위와 같다. 이때 출력 형태를 Berkeley 스타일로 출력하고 싶으면 --format=berkeley 말고 -B 옵션을 채택해도 된다. 마찬가지로 출력 형태를 System V 스타일로 출력하고 싶으면 --format=sysv 말고 -A 옵션을 채택해도 된다.

## 11. strings → print the strings of printable characters in files

strings 명령어는 바이너리 파일에서 사람이 읽을 수 있는 문자열만 추출해주는 도구이다. 목적으로는 바이너리 안에 포함된 ASCII 문자열을 찾는 것이고, 용도로는 디버깅, 보안 분석, 리버싱, 숨겨진 메시지 찾기, 라이브러리 의존성 확인 등이 있다.

```

user@ubuntu:~/test11$ strings hello
/lib64/ld-linux-x86-64.so.2
]?gZ
libc.so.6
puts
__cxa_finalize
__libc_start_main
GLIBC_2.2.5
__ITM_deregisterTMCloneTable
__gmon_start__
__ITM_registerTMCloneTable
AWAVI
AUATL
[]A\A]A^A_
Hello, strings!
;*3$"
GCC: (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
crtstuff.c
deregister_tm_clones
__do_global_ctors_aux
completed.7698
__do_global_ctors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
hello.c
__FRAME_END__
__init_array_end
_DYNAMIC
__init_array_start
GNU_EH_FRAME_HDR
_GLOBAL_OFFSET_TABLE_
__libc_csu_fini
__ITM_deregisterTMCloneTable
puts@GLIBC_2.2.5
edata
__libc_start_main@@GLIBC_2.2.5
data_start
__gmon_start__
dso_handle
IO_stdin_used
__libc_csu_init
bss_start
main
__TMC_END__
__ITM_registerTMCloneTable
__cxa_finalize@@GLIBC_2.2.5
.symtab
.strtab

```

위 사진은 실행 파일 hello에서 문자열을 추출한 것으로, 코드에 사용한 문자열, 동적 링크 관련 문자열, 링커가 포함한 내부 문자열 등을 추출한다.

```

user@ubuntu:~/test11$ strings -n 6 hello
/lib64/ld-linux-x86-64.so.2
libc.so.6
__cxa_finalize
__libc_start_main
GLIBC_2.2.5
__ITM_deregisterTMCloneTable
__gmon_start__
__ITM_registerTMCloneTable
[]A\A]A^A_
Hello, strings!
GCC: (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
crtstuff.c
deregister_tm_clones
__do_global_ctors_aux
completed.7698
__do_global_ctors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
hello.c
__FRAME_END__
__init_array_end
__DYNAMIC
__init_array_start
__GNU_EH_FRAME_HDR
__GLOBAL_OFFSET_TABLE__
__libc_csu_fini
__ITM_deregisterTMCloneTable
puts@@GLIBC_2.2.5
edata
__libc_start_main@@GLIBC_2.2.5
data_start
__gmon_start__
dso_handle
IO_stdin_used
__libc_csu_init
bss_start
__TMC_END__
__ITM_registerTMCloneTable
__cxa_finalize@@GLIBC_2.2.5
.symtab
.strtab
.shstrtab
.interp
.note.ABI-tag
.note.gnu.build-id
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r

```

위 사진은 최소 길이를 지정해 출력한 것으로, 6자 이상인 문자열만 보여준다. 이때, 기본 값은 4 글자 이상이다.

```

user@ubuntu:~/test11$ strings -t x hello
238 /lib64/ld-linux-x86-64.so.2
28f ]?gZ
361 libc.so.6
36b puts
370 __cxa_finalize
37f __libc_start_main
391 GLIBC_2.2.5
39d _ITM_deregisterTMCloneTable
3b9 __gmon_start__
3c8 _ITM_registerTMCloneTable
660 AWAVI
667 AUATL
6ba [JA\A]A^A_
6e4 Hello, strings!
797 ;*3$
1010 GCC: (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
1629 crtstuff.c
1634 deregister_tm_clones
1649 __do_global_ctors_aux
165f completed.7698
166e __do_global_ctors_aux_fini_array_entry
1695 frame_dummy
16a1 __frame_dummy_init_array_entry
16c0 hello.c
16c8 __FRAME_END__
16d6 __init_array_end
16e7 _DYNAMIC
16f0 __init_array_start
1703 __GNU_EH_FRAME_HDR
1716 _GLOBAL_OFFSET_TABLE_
172c __libc_csu_fini
173c _ITM_deregisterTMCloneTable
1758 puts@@GLIBC_2.2.5
176a _edata
1771 __libc_start_main@@GLIBC_2.2.5
1790 __data_start
179d __gmon_start__
17ac __dso_handle
17b9 _IO_stdin_used
17c8 __libc_csu_init
17d8 __bss_start
17e4 main
17e9 __TMC_END__
17f5 _ITM_registerTMCloneTable
180f __cxa_finalize@@GLIBC_2.2.5
182c .symtab
1834 .strtab
183c .shstrtab
1846 .interp
184e .note.ABI-tag

```

위 사진은 오프셋과 함께 보는 것으로, 문자열의 파일 내 오프셋(offset)을 16진수로 표시한다.

```

user@ubuntu:~/test11$ strings hello | grep Hello
Hello, strings!

```

위 사진은 특정 문자열 포함 여부를 확인하는 것으로, grep 명령어를 써서 특정 문자열이 포함되어 있는 지 확인할 수 있다.

```

user@ubuntu:~/test11$ strings -f hello
hello: /lib64/ld-linux-x86-64.so.2
hello: ]?gZ
hello: libc.so.6
hello: puts
hello: __cxa_finalize
hello: __libc_start_main
hello: GLIBC_2.2.5
hello: _ITM_deregisterTMCloneTable
hello: __gmon_start__
hello: _ITM_registerTMCloneTable
hello: AWAVI
hello: AUATL
hello: [JA\A]A^A_
hello: Hello, strings!
hello: ;*3$"
hello: GCC: (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
hello: crtstuff.c
hello: deregister_tm_clones
hello: __do_global_dtors_aux
hello: completed.7698
hello: __do_global_dtors_aux_fini_array_entry
hello: frame_dummy
hello: __frame_dummy_init_array_entry
hello: hello.c
hello: __FRAME_END__
hello: __init_array_end
hello: __DYNAMIC
hello: __init_array_start
hello: __GNU_EH_FRAME_HDR
hello: __GLOBAL_OFFSET_TABLE__
hello: __libc_csu_fini
hello: _ITM_deregisterTMCloneTable
hello: puts@@GLIBC_2.2.5
hello: .edata
hello: __libc_start_main@@GLIBC_2.2.5
hello: __data_start
hello: __gmon_start__
hello: __dso_handle
hello: _IO_stdin_used
hello: __libc_csu_init
hello: .bss.start

```

위 사진은 -f 옵션으로 각 문자열 행 앞에 파일 이름을 같이 출력해준다.

## 12. strip → Discard symbols from object files

strip 명령어는 ELF 파일에서 디버깅 심볼, 심볼 테이블 등에서 불필요한 정보를 제거하기 위한 용도로 쓰인다.

```

user@ubuntu:~/test12$ ls -lh hello
-rwxrwxr-x 1 user user 11K Apr 10 14:20 hello
user@ubuntu:~/test12$ nm hello | grep greet
0000000000000063a T greet
user@ubuntu:~/test12$ objdump -t hello | grep greet
0000000000000063a g      F .text 0000000000000013      greet
user@ubuntu:~/test12$ strip hello
user@ubuntu:~/test12$ nm hello | grep greet
nm: hello: no symbols
user@ubuntu:~/test12$ ls -lh hello
-rwxrwxr-x 1 user user 6.0K Apr 10 14:21 hello

```

초기에 ls -lh를 통해 파일 크기를 비교할 수 있게 하고, nm 또는 objdump 명령어를 사용하여 심볼 테이블을 확인한다. 이후에 strip 명령어를 사용해서 hello에서 모든 디버깅 정보, 심볼 테이블을 삭제한다. 그리고 나서 다시 nm 명령어로 확인해 보면 삭제되어 아무것도 출력이 되지 않는 것을 확인할 수 있다. 또한 다시 ls -lh 명령을 통해 크기가 줄어든 것도 확인할 수 있다.

```
user@ubuntu:~/test12$ gcc -g -o hello hello.c
user@ubuntu:~/test12$ cp hello hello_debug
user@ubuntu:~/test12$ strip --strip-debug hello_debug
user@ubuntu:~/test12$ nm hello_debug | grep greet
0000000000000063a T greet
```

다시 처음으로 돌아가서 hello 실행 파일을 생성한 후에 strip 명령어에 디버그 정보만 제거할 수 있게 --strip-debug를 사용하고 nm으로 심볼을 출력해보면 함수 심볼(greet, main)이 남아 있는 것을 확인할 수 있다.

```
user@ubuntu:~/test12$ cp hello hello_nogreet
user@ubuntu:~/test12$ strip --strip-symbol=greet hello_nogreet
user@ubuntu:~/test12$ nm hello_nogreet | grep greet
user@ubuntu:~/test12$
```

이번엔 strip 명령어에 특정 심볼 테이블만 제거할 수 있게 --strip-symbol=greet을 사용하고 nm으로 심볼을 출력해보면 제거된 것을 확인할 수 있다.